

PSP

UD5. PROGRAMACIÓN SEGURA

PRACTICA DE CLASE 4

JWT

Los Tokens JWT (<https://jwt.io/>) son un estándar abierto que permite representar información en un formato seguro y compacto.

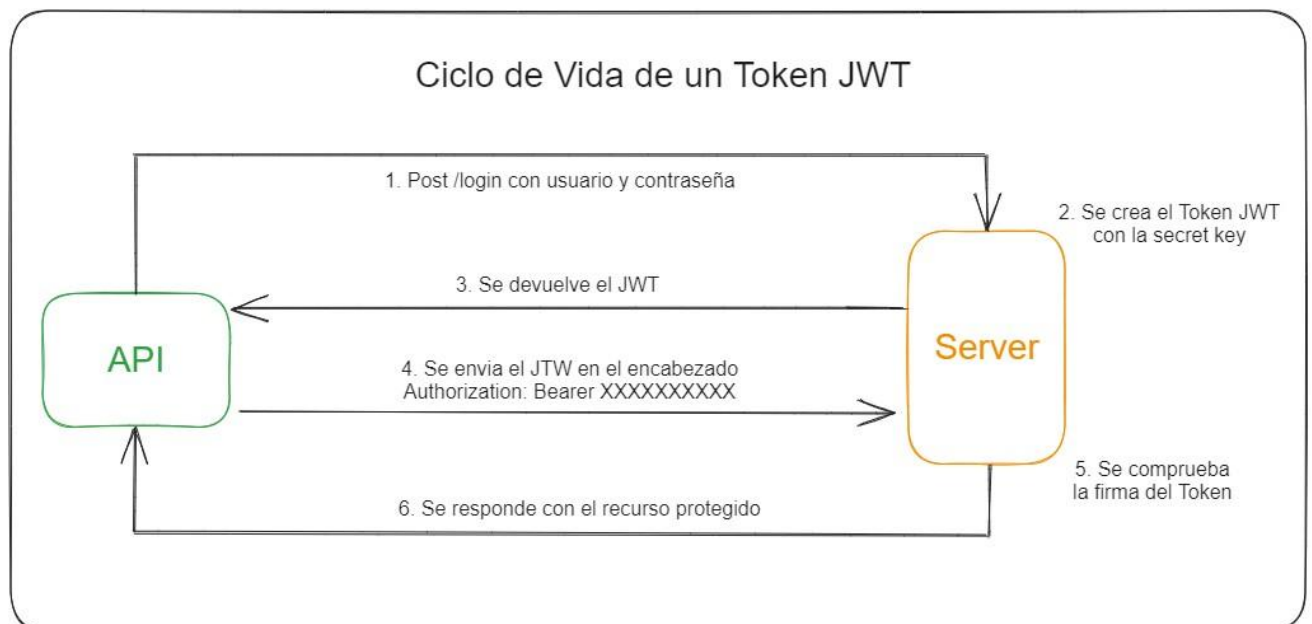
Estos tokens son firmados digitalmente y pueden contener datos como la identidad del usuario, roles, permisos y otra información relevante para la autenticación.

Estructura de un JWT

El token JWT (JSON Web Token) está compuesto por tres secciones: El encabezado (header), la carga útil (payload) y la firma (signature).

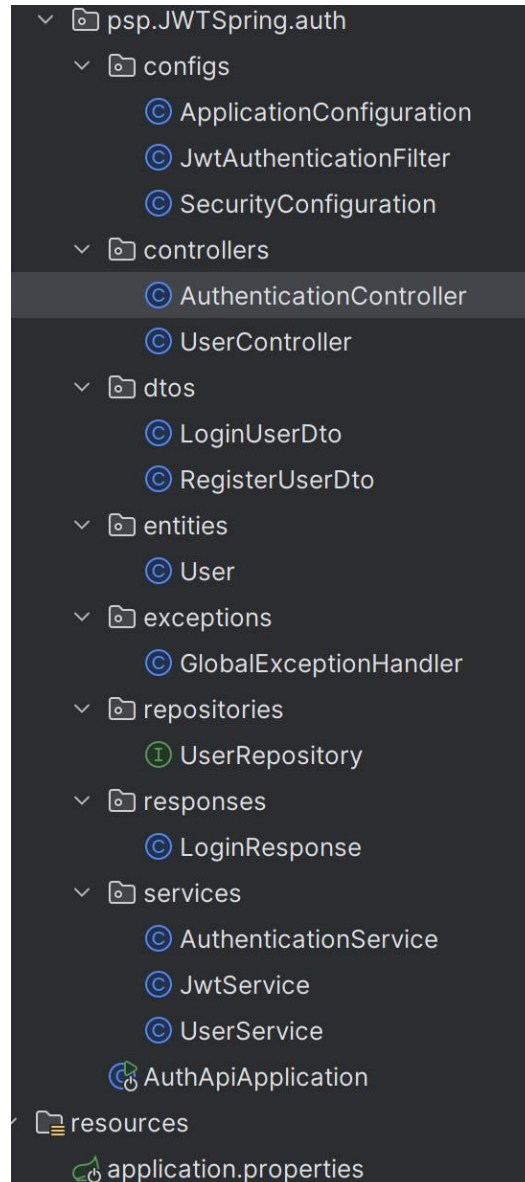
- **Header:** Contiene información sobre el tipo de token y el algoritmo de firma.
- **Payload:** Almacena datos del usuario y otra información adicional en formato JSON.
- **Signature:** Garantiza la integridad y autenticidad del token mediante una clave secreta o privada.

Ciclo de vida de un JWT



CREACIÓN DE API DE AUTORIZACION CON JWT

Vamos a crear un proyecto de una API de autorización con Spring Boot y JWT. El proyecto, tiene la siguiente estructura:



Breve introducción:

Proceso de Generación y Verificación de Tokens JWT

1. Generación del Token:

- Cuando un usuario se autentica correctamente, el servidor genera un token JWT.
- Este token se firma utilizando la clave secreta.
- El token firmado se envía al cliente. Este token contiene información como el nombre de usuario y la fecha de expiración, y está firmado para asegurar su integridad.

2. Envío del Token al Cliente:

- El token JWT firmado se envía al cliente en la respuesta de la autenticación.
- El cliente almacena este token (por ejemplo, en el almacenamiento local o en las cookies) y lo incluye en las cabeceras de las solicitudes posteriores al servidor.

3. Verificación del Token:

- Cuando el cliente realiza una solicitud al servidor, incluye el token JWT en la cabecera de autorización.
- El servidor recibe el token y utiliza la misma clave secreta (`security.jwt.secretkey`) para verificar la firma del token.
- Si la firma es válida y el token no ha expirado, el servidor procesa la solicitud. Si no, el servidor rechaza la solicitud.

API

Empezamos por `application.properties`:

`spring.datasource.driver-class-name=org.mariadb.jdbc.Driver server.port=8085`

`spring.datasource.url=jdbc:mariadb://localhost:3306/apirest`

`spring.datasource.username=root spring.datasource.password=root`

`spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect spring.jpa.show-sql=true`

`spring.jpa.properties.hibernate.format_sql=true`

`## Hibernate properties spring.jpa.hibernate.ddl-auto=update`

`spring.jpa.open-in-view=false`

`security.jwt.secret-`

`key=3cfa76ef14937c1c0ea519f8fc057a80fcd04a7420f8e8bcd0a7567c272e007b`

`# 24h in millisecond`

`security.jwt.expiration-time=86400000`

La api está completa excepto `JWTAuthenticationFilter` y `SecurityConfiguration` que debeis implementar con el siguiente código, explicando en ambas qué hace cada función y/o método. (A través de comentarios)

JwtAuthenticationFilter.java

```
package psp.JWTSpring.auth.configs; import
    psp.JWTSpring.auth.services.JwtService;
import jakarta.servlet.FilterChain; import
    jakarta.servlet.ServletException; import
    jakarta.servlet.http.HttpServletRequest; import
    jakarta.servlet.http.HttpServletResponse; import
    org.springframework.lang.NonNull;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication; import
    org.springframework.security.core.context.SecurityContextHolder; import
    org.springframework.security.core.userdetails.UserDetails; import
    org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component; import
    org.springframework.web.filter.OncePerRequestFilter; import
    org.springframework.web.servlet.HandlerExceptionResolver; import java.io.IOException;
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {    private
    final HandlerExceptionResolver handlerExceptionResolver;

    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    public JwtAuthenticationFilter(
        JwtService jwtService,
        UserDetailsService userDetailsService,
        HandlerExceptionResolver handlerExceptionResolver
    ){
        this.jwtService = jwtService;
        this.userDetailsService = userDetailsService;
        this.handlerExceptionResolver = handlerExceptionResolver;
    }

    @Override
    protected void doFilterInternal(
        @NonNull HttpServletRequest request,
        @NonNull HttpServletResponse response,
        @NonNull FilterChain filterChain
    ) throws ServletException, IOException {
        final String authHeader = request.getHeader("Authorization");

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);    return;
        }
    }
}
```

```
try {
    final String jwt = authHeader.substring(7);
    final String userEmail = jwtService.extractUsername(jwt);

    Authentication authentication = SecurityContextHolder.getContext().getAuthentication();

    if (userEmail != null && authentication == null) {
        UserDetails userDetails = this.userDetailsService.loadUserByUsername(userEmail);
        if (jwtService.isTokenValid(jwt, userDetails)) {
            UsernamePasswordAuthenticationToken authToken = new
            UsernamePasswordAuthenticationToken(
                userDetails,
                null,
                userDetails.getAuthorities()
            );

            authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
            SecurityContextHolder.getContext().setAuthentication(authToken);
        }
    }

    filterChain.doFilter(request, response);
} catch (Exception exception) {
    handlerExceptionResolver.resolveException(request, response, null, exception);
}
}
```

SecurityConfiguration.java

```
package psp.JWTSpring.auth.configs;

import org.springframework.context.annotation.Bean; import
org.springframework.context.annotation.Configuration; import
org.springframework.security.authentication.AuthenticationProvider; import
org.springframework.security.config.annotation.web.builders.HttpSecurity; import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity; import
org.springframework.security.config.http.SessionCreationPolicy; import
org.springframework.security.web.SecurityFilterChain; import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration; import
org.springframework.web.cors.CorsConfigurationSource; import
org.springframework.web.cors.UrlBasedCorsConfigurationSource; import java.util.List;
```

```

@Configuration
@EnableWebSecurity
public class SecurityConfiguration {    private final
AuthenticationProvider authenticationProvider;    private final
JwtAuthenticationFilter jwtAuthenticationFilter;

    public SecurityConfiguration(
        JwtAuthenticationFilter jwtAuthenticationFilter,
        AuthenticationProvider authenticationProvider
    ){
        this.authenticationProvider = authenticationProvider;
        this.jwtAuthenticationFilter = jwtAuthenticationFilter;
    }
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http

            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/auth/**").permitAll()
                .anyRequest().authenticated()
            )
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
            )
            .authenticationProvider(authenticationProvider)
            .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
    @Bean
    CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration configuration = new CorsConfiguration();

        configuration.setAllowedOrigins(List.of("http://localhost:8005"));
        configuration.setAllowedMethods(List.of("GET", "POST"));
        configuration.setAllowedHeaders(List.of("Authorization", "Content-Type"));

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", configuration);

        return source;
    }
}

```

PRUEBAS:**En Postman:**

Tu servidor está configurado en el puerto 8085, así que la URL base es `http://localhost:8085`.

1. Registrar un Usuario (Sign Up)

Primero creamos un usuario.

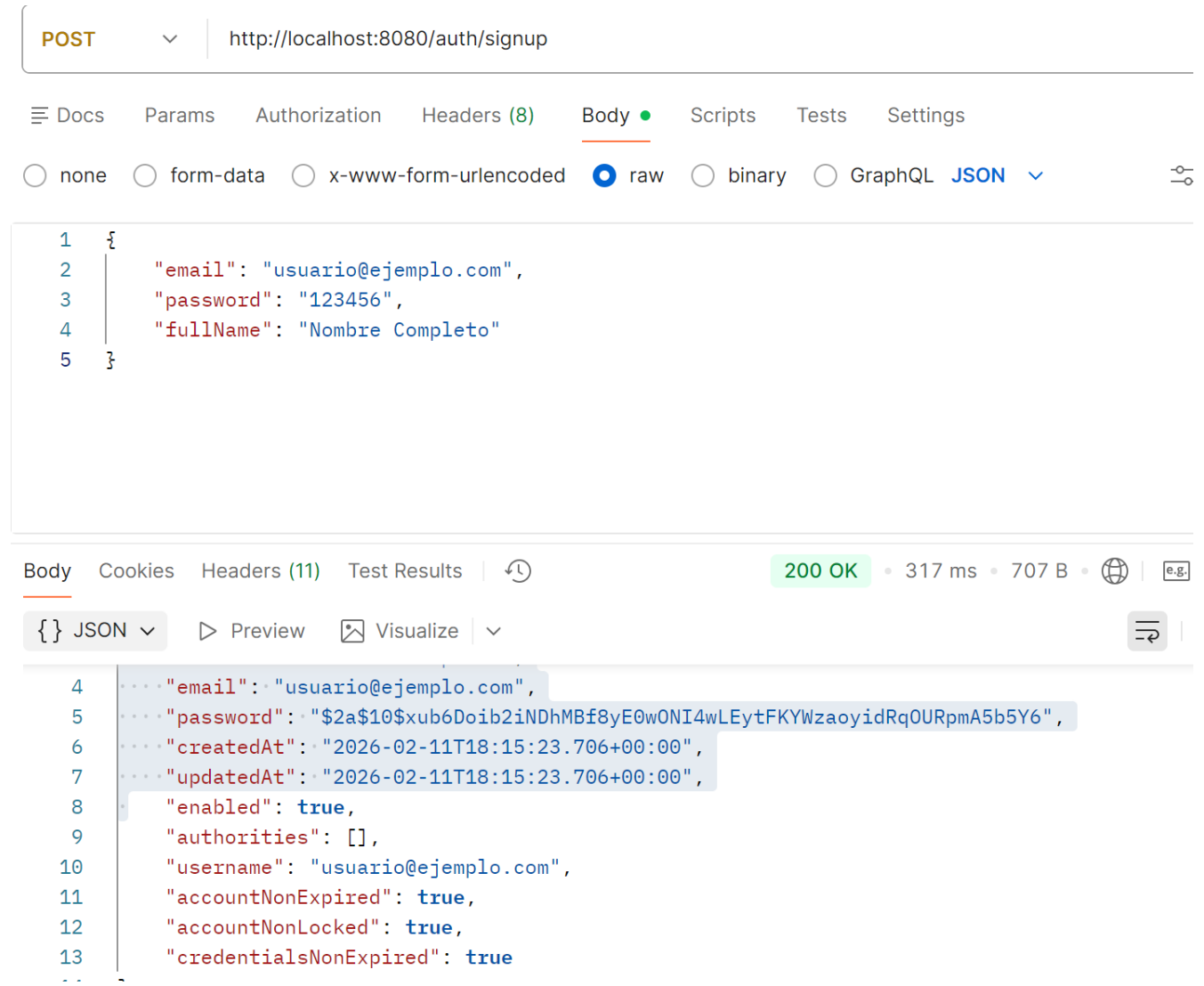
Método: POST

URL: `http://localhost:8085/auth/signup`

Body (Cuerpo): Seleccionamos la pestaña Body, luego raw y eligimos JSON en el desplegable de la derecha.

JSON a enviar:

```
{
  "email": "usuario@ejemplo.com",
  "password": "tuPassword123",
  "fullName": "Nombre Completo"
}
```



2. Iniciar Sesión (Login)

Una vez registrado, usamos este endpoint para obtener nuestro Token JWT.

Método: POST

URL: http://localhost:8085/auth/login

Body (Cuerpo): Pestaña Body -> raw -> JSON.

JSON a enviar:

```
{
  "email": "usuario@ejemplo.com",
  "password": "tuPassword123"
}
```

Respuesta: Recibimos un JSON con el campo token. Copiamos este token, lo necesitaremos para las siguientes peticiones.

The screenshot shows a REST client interface. At the top, the method is set to POST and the URL is http://localhost:8080/auth/login. Below the URL bar, there are tabs for Docs, Params, Authorization, Headers (8), Body, Scripts, Tests, and Settings. The Body tab is selected, and the format is set to raw JSON. The request body is a JSON object with the following content:

```
1 {
2   "email": "usuario@ejemplo.com",
3   "password": "123456"
4 }
```

Below the request body, the response is shown. The status is 200 OK, with a response time of 479 ms and a body size of 516 B. The response body is a JSON object with the following content:

```
1 {
2   "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2Vhcm1vQG9vZmVqZW1wbG8uY29tIiwiaWF0IjoxNzAwODMzNzY3LCJleHAiOjE3NzA5MjAxNzY3LmVhcnQum2hMmsgQ8nxZN-0jplRWzm0Gk7DboxM",
3   "expiresIn": 86400000
4 }
```

3. Accedemos a las rutas protegidas (Ejemplo: Obtener usuario actual)

Para acceder a cualquier otro endpoint (como los definidos en UserController), necesitamos enviar el token.

Método: GET

URL: http://localhost:8080/users/me

Authorization (Autorización):

Vamos a la pestaña Auth (o Authorization).

- En "Type" seleccionamos Bearer Token.
- En el campo "Token", pegamos el token que copiamos en el paso anterior (el del Login).
- Enviar: Al darle a "Send", deberíamos ver los datos de tu usuario.

GET http://localhost:8080/users/me

Send

Docs Params Authorization Headers (9) Body Scripts Tests Settings Cookies

Auth Type: Bearer Token

Token: eyJhbGciOiJIUzI1NiJ9.eyJzdWliOiJ1c2...

Body Cookies Headers (11) Test Results 200 OK 30 ms 707 B Save Response

```

1 {
2   "id": 52,
3   "fullName": "Nombre Completo",
4   "email": "usuario@ejemplo.com",
5   "password": "$2a$10$xub6Doib2iNDhMBf8yE0wONI4wLEytFKYWzaoyidRq0URpmA5b5Y6",
6   "createdAt": "2026-02-11T18:15:23.706+00:00",
7   "updatedAt": "2026-02-11T18:15:23.706+00:00",
8   "enabled": true,
9   "authorities": [],
10  "username": "usuario@ejemplo.com",
11  "accountNonExpired": true,
12  "accountNonLocked": true,
13  "credentialsNonExpired": true
14 }
```

EXPLICACION DEL CÓDIGO

1. ApplicationConfiguration — El núcleo de autenticación

Esta clase define cómo Spring va a autenticar usuarios.

Está anotada con:

@Configuration

→ Indica que es una clase de configuración de Spring (crea Beans).

✓ UserDetailsService

@Bean

```

UserDetailsService userDetailsService() {
    return username -> userRepository.findByEmail(username)
        .orElseThrow(() -> new UsernameNotFoundException("Usuario no encontrado"));
}
```

Spring Security NO sabe cómo están guardados tus usuarios.

“Cuando alguien intente loguearse, búscalo en la BD por email”.

Flujo real:

1. Usuario introduce email + password.
2. Spring llama a UserDetailsService.
3. Se consulta UserRepository.
4. Si no existe → excepción.

Esto es el puente entre Spring Security y tu base de datos.

✓ PasswordEncoder

@Bean

```
BCryptPasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Nunca se debe guardar passwords en texto plano.

BCrypt:

- añade salt
- es lento (protege contra brute force)
- es estándar en Spring

✓ AuthenticationManager

@Bean

```
public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws Exception {
    return config.getAuthenticationManager();
}
```

Es el motor que ejecuta la autenticación.

Cuando hacemos login:

Controller → AuthenticationManager → AuthenticationProvider.

No valida directamente — delega.

✓ AuthenticationProvider

@Bean

```
AuthenticationProvider authenticationProvider() {
    DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider();

    authProvider.setUserDetailsService(userDetailsService());
    authProvider.setPasswordEncoder(passwordEncoder());

    return authProvider;
}
```

El DaoAuthenticationProvider:

- ✓ busca el usuario
- ✓ compara passwords
- ✓ crea el Authentication

Flujo interno:

Login request



AuthenticationProvider



UserDetailsService



PasswordEncoder.matches()

2. JwtAuthenticationFilter — El guardia de cada request

Extiende:

OncePerRequestFilter

Se ejecuta una vez por cada petición HTTP.

Paso a paso del filtro

1 Leer header

```
final String authHeader = request.getHeader("Authorization");
```

Busca:

Authorization: Bearer eyJhbGciOiJIUzI1NiIs...

Si no existe:

```
filterChain.doFilter(request, response);
```

deja pasar la request sin autenticar.

2 Extraer token

```
final String jwt = authHeader.substring(7);
```

Quita "Bearer ".

3 Obtener usuario del token

```
final String userEmail = jwtService.extractUsername(jwt);
```

El JWT contiene claims.

4 Evitar re-autenticación

```
Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
```

Si ya está autenticado → no repite.

5 Validar token

```
if (jwtService.isTokenValid(jwt, userDetails))
```

Debe comprobar:

✓ firma

✓ expiración

✓ integridad

6 Crear identidad en Spring

```
UsernamePasswordAuthenticationToken authToken =
```

```
    new UsernamePasswordAuthenticationToken(
        userDetails,
        null,
        userDetails.getAuthorities()
    );
```

Esto le dice a Spring:

“Este usuario ya es de confianza”.

Luego:

```
SecurityContextHolder.getContext().setAuthentication(authToken);
```

ESTE es el momento exacto en que el usuario queda autenticado.

3. SecurityConfiguration — El muro de seguridad

Activa Spring Security:

`@EnableWebSecurity`

✓ **SecurityFilterChain**

Este método define las reglas de seguridad.

CSRF

`.csrf(csrf -> csrf.disable())`

Correcto para APIs REST sin sesión.

Rutas públicas

`.requestMatchers("/auth/**").permitAll()`

Permite:

- login
- register

Si no → nadie podría autenticarse

Todo lo demás requiere login

`.anyRequest().authenticated()`

Sin sesión

`.sessionCreationPolicy(SessionCreationPolicy.STATELESS)`

CRÍTICO para JWT.

Significa:

El servidor NO guarda sesiones.

Cada request debe traer su token.

Registrar provider

`.authenticationProvider(authenticationProvider)`

Le dice a Spring qué motor usar.

Insertar el filtro JWT

`.addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);`

El JWT debe ejecutarse ANTES del filtro de login clásico.

Porque:

- si el token es válido
- no hace falta login

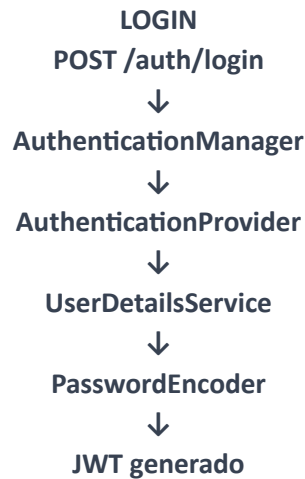


CORS
`configuration.setAllowedOrigins(List.of("http://localhost:8005"));`
 Permite llamadas desde ese frontend.

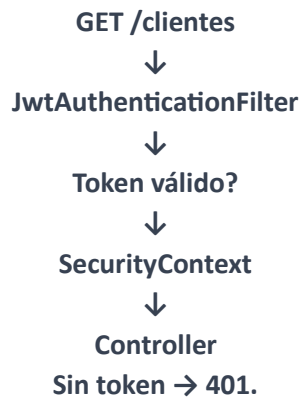
En producción normalmente:

- dominio real
- no localhost

Cómo funciona TODO junto (flujo real)



REQUEST PROTEGIDA



CONTESTAD A LAS SIGUIENTES PREGUNTAS:

1. ¿Cómo puedo hacer que el token dure como máximo 1 hora?
2. ¿Qué áreas son sin autorización y cuales con autorización?
3. Explica brevemente cómo funciona la API, según la llamada que se realice.
4. Encuentra dentro del código qué función hash está utilizando para firmar el token y busca información sobre ella.
5. Encuentra dentro del código qué función se utiliza al realizar un signup y almacena el hash de la contraseña.